

Filtro de Sobel en Python: Comparación secuencial, NumPy, Numba CPU y Numba GPU

Institución: UNTDF

Carrera: Lic. en Sistemas, 5to año

Materia: Sistemas Paralelos

Docente: MsC. Federico González Brizzio

Abstract

Esta segunda entrega agrega un único caso nuevo al trabajo previo: `numba_gpu`. Los resultados de `secuencial`, `numpy` y `numba_cpu` se copian desde `resultados_sobel_entrega1.csv`, generado en la entrega 1, y luego se combinan con `resultados_sobel_entrega2.csv`. La implementación CUDA sigue la estructura didáctica del material de cátedra: `kernels @ cuda.jit`, `cuda.grid(2)`, un hilo por píxel y recorrido local 3x3 para Sobel. La comparación principal se realiza contra el secuencial usando el tiempo total, reportando tanto speed-up como mejora porcentual.

1. Introducción

El operador Sobel calcula un gradiente local para cada píxel usando una vecindad 3x3. En GPU, esa independencia permite asignar un píxel por thread CUDA. Esta entrega no reejecuta los casos de la entrega anterior; solo incorpora la versión `numba_gpu` y la compara contra los resultados ya obtenidos.

2. Metodología

2.1 Equipo

Propiedad	Valor
CPU	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Núcleos lógicos	8
GPU	NVIDIA GeForce GTX 1650
Multiprocesadores CUDA reportados	14
Python	3.14.5
NumPy	2.4.4
Numba	0.65.1
Pillow	12.2.0

2.2 Algoritmos incluidos

1. **Secuencial**: resultado de la entrega 1.
2. **NumPy**: resultado de la entrega 1.
3. **Numba CPU**: resultado de la entrega 1.
4. **Numba GPU**: caso nuevo de la entrega 2.

2.3 Parámetros experimentales

Parámetro	Valor
Tamaños	750x750, 1500x1500, 3000x3000, 6000x6000
Corridas por caso	5
Resultados base	/mnt/sda1/code/facu/paralelos/t p3.1/resultados_sobel_entrega1. csv
Resultado nuevo	/mnt/sda1/code/facu/paralelos/t p3.2/resultados_sobel_entrega2. csv
Script de corrida GPU	python correr_entrega2.py

La carga de imágenes y el guardado de salidas no forman parte de las mediciones.

2.4 Métricas

- **Tiempo RGB->gris (s):** tiempo promedio de conversión.
 - **Tiempo Sobel (s):** tiempo promedio del filtro Sobel.
 - **Tiempo total (s):** suma medida de conversión y Sobel.
 - **% blancos:** (píxeles con valor 255 / píxeles totales) * 100.
 - **Speed-up vs secuencial:** tiempo_total_secuencial / tiempo_total_metodo.
 - **Mejora vs secuencial (%):** (1 - tiempo_total_metodo / tiempo_total_secuencial) * 100.
-

3. Comparación combinada

Las tablas siguientes integran los tres casos de la entrega 1 con el caso nuevo numba_gpu. La mejora porcentual se calcula siempre respecto del secuencial del mismo tamaño.

3.1 Imagen 750x750

Método	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up vs secuencial	Mejora vs secuencial (%)
secuencial	0.13759 8206	0.38108 0508	0.51867 8714	0.28177 7778	1.00000 0	0.00
numpy	0.00358 4823	0.00527 3254	0.00885 8076	0.28177 7778	58.5543 31	98.29

Método	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up vs secuencial	Mejora vs secuencial (%)
numba_cpu	0.008086321	0.001361815	0.009448136	0.281777778	54.897465	98.18
numba_gpu	0.001899755	0.000691792	0.002591548	0.281777778	200.142430	99.50

3.2 Imagen 1500x1500

Método	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up vs secuencial	Mejora vs secuencial (%)
secuencial	0.589171745	1.607831727	2.197003472	0.059955556	1.000000	0.00
numpy	0.013785413	0.025855008	0.039640422	0.059955556	55.423312	98.20
numba_cpu	0.001911381	0.005410280	0.007321660	0.059955556	300.069038	99.67
numba_gpu	0.003172580	0.002474146	0.005646725	0.059955556	389.075698	99.74

3.3 Imagen 3000x3000

Método	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up vs secuencial	Mejora vs secuencial (%)
secuencial	2.425503625	6.512637727	8.938141351	0.001366667	1.000000	0.00
numpy	0.060008272	0.120068416	0.180076688	0.001366667	49.635194	97.99
numba_cpu	0.006325961	0.011174299	0.017500260	0.001366667	510.743346	99.80
numba_gpu	0.010337396	0.007141225	0.017478621	0.001366667	511.375660	99.80

3.4 Imagen 6000x6000

Método	Tiempo RGB->grayscale (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up vs secuencial	Mejora vs secuencial (%)
secuencial	9.813729013	26.460540787	36.274269800	0.000000000	1.000000	0.00
numpy	1.742763535	2.402780660	4.145544195	0.000000000	8.750183	88.57
numba_cpu	0.027100780	0.029705952	0.056806732	0.000000000	638.555828	99.84
numba_gpu	0.039567730	0.071842688	0.111410418	0.000000000	325.591363	99.69

4. Discusión

4.1 Mejora porcentual de Numba GPU contra secuencial

La mejora de numba_gpu respecto del secuencial, usando tiempo total, fue: 99.50% en 750x750 (200.14x), 99.74% en 1500x1500 (389.08x), 99.80% en 3000x3000 (511.38x), 99.69% en 6000x6000 (325.59x).

Esta es la comparación central de la entrega: el nuevo caso GPU se evalúa contra la base secuencial del trabajo original.

4.2 Amortización de transferencias CPU<->GPU

En 6000x6000, las transferencias GPU suman aproximadamente 0.06375 s (H2D + D2H) y los kernels suman 0.04766 s. El tiempo total GPU es 0.11141 s frente a 36.27427 s del secuencial. La mejora se observa cuando el paralelismo de la GPU compensa el costo de copiar datos entre host y dispositivo.

4.3 Consistencia de salida

Tamaño	Secuencial % blancos	NumPy % blancos	Numba CPU % blancos	Numba GPU % blancos
750x750	0.281777778	0.281777778	0.281777778	0.281777778
1500x1500	0.059955556	0.059955556	0.059955556	0.059955556
3000x3000	0.001366667	0.001366667	0.001366667	0.001366667

Tamaño	Secuencial % blancos	NumPy % blancos	Numba CPU % blancos	Numba GPU % blancos
6000x6000	0.0000000000	0.0000000000	0.0000000000	0.0000000000

Los porcentajes coinciden entre los cuatro métodos para cada tamaño, por lo que la comparación es de rendimiento y no de diferencia en la métrica de salida.